Code Explanation : MINICPU.ASM, line by line

Use this to answer **"How did you make it?"** and **"What does this line do?"** The line numbers match `src/MINICPU.ASM` as it appears in the emu8086 editor.

---

1. How we made it (say this first)

> "We made a small imaginary CPU inside an 8086 program.
>
> 1. The **memory** of the small CPU is a 16-byte array called `ram`.
> 2. Its **registers** (PC, MAR, MDR, IR, ACC) are 1-byte variables.
> 3. A **loop** calls three procedures again and again, **fetch**, **decode** and **execute**, until the CPU reaches HALT.
> 4. Every step prints the registers on the screen using **INT 21h**, so we can watch the CPU work."

Part of the small CPU	How it is made in 8086 code
Memory (16 bytes)	`ram db ...` (an array of 16 bytes)
Registers PC, MAR, MDR, IR, ACC	`cpu_pc`, `cpu_mar`, `cpu_mdr`, `cpu_ir`, `cpu_acc` (byte variables)
Instruction cycle	the `cpu_cycle` loop
Fetch, Decode, Execute	three procedures: `fetch`, `decode`, `execute`
ALU (add, subtract)	the 8086's own `ADD` and `SUB` instructions
Output on screen	INT 21h: function 09h prints a message, 02h prints one character
Key press	INT 21h, function 07h

---

2. The code, block by block

Lines 1 to 28 : Header comment

Lines starting with `;` are comments, which the assembler ignores. They describe the small CPU and its 6 instructions.

Line 30 : `org 100h`

This tells the assembler the program is a **.COM program**, which DOS loads at address **100h**. emu8086 uses this to build a COM file.

Line 32 : `jmp start`

The data (variables and messages) comes first in the file, so the CPU must **jump over the data** to the first real instruction, at the `start:` label.

Lines 50 to 51 : The memory, `ram`

```
ram     db 1Dh, 2Eh, 3Fh, 4Ch, 56h, 2Eh, 00h, 00h
        db 00h, 00h, 00h, 00h, 00h, 09h, 05h, 03h
```

- `db` means **define byte**. This creates 16 bytes, at addresses 00 to 0F.
- The bytes at addresses 00 to 06 are the **program**:
- `1D` = LOAD 0D
- `2E` = ADD 0E
- `3F` = SUB 0F
- `4C` = STORE 0C
- `56` = JMP 06
- `2E` = ADD 0E (skipped)
- `00` = HALT
- The bytes at addresses 0D, 0E and 0F are the **data**: 9, 5 and 3.

Lines 56 to 64 : The registers

```
cpu_pc   db 0     ; Program Counter
cpu_mar  db 0     ; Memory Address Register
...
```

Each register of the small CPU is a 1-byte variable that starts at 0.

- `op_code` and `op_addr` hold the two parts of the decoded instruction.
- `halted` becomes 1 when the CPU executes HALT.

Lines 69 to 99 : Messages

- Text to print, such as `'[FETCH] ...'`.
- `13, 10` = carriage return + line feed, which moves to a new line.
- `'$'` marks the **end of the message**. INT 21h function 09h prints up to the `$`.

Lines 104 to 107 : Start

```
lea dx, msg_title    ; DX = address of the message
call print_str       ; print it
call show_memory     ; print the 16 bytes of memory
```

`lea` (load effective address) puts the **address** of a message into DX, which is what INT 21h function 09h needs.

Lines 109 to 124 : The main loop, `cpu_cycle`

```
 call wait_key  /  call fetch      ; step 1
call wait_key  /  call decode     ; step 2
call wait_key  /  call execute    ; step 3
mov al, halted
cmp al, 1        ; was HALT executed?
jne cpu_cycle    ; if not, repeat for the next instruction
```

This is the **instruction cycle**. Each step waits for a key so we can watch it. The loop stops when `halted` is 1.

Lines 127 to 140 : End of the program

- Prints "CPU HALTED" and the result stored at memory address 0C (`ram[0Ch]`).
- Prints the memory again, so the changed byte at 0C is visible.
- Waits for a key, then exits to DOS with **INT 21h, function 4Ch**.

Lines 145 to 164 : FETCH

Line	Code	Meaning
146 to 147	`mov al, cpu_pc` / `mov cpu_mar, al`	**MAR ← PC**
149 to 150	`mov bl, cpu_mar` / `mov bh, 0`	BX = the address (BH must be 0, so BX = BL)
151 to 152	`mov al, ram[bx]` / `mov cpu_mdr, al`	**MDR ← M[MAR]**: read memory
154	`mov cpu_ir, al`	**IR ← MDR**
156 to 159	`inc al` / `and al, 0Fh` / `mov cpu_pc, al`	**PC ← PC + 1** (the AND keeps PC between 00 and 0F)
161 to 163	print the message and the registers	

Lines 169 to 189 : DECODE

Line	Code	Meaning
170 to 173	`mov al, cpu_ir` / `mov cl, 4` / `shr al, cl`	Shift IR right by 4 bits, which leaves the **opcode** (high 4 bits). Example: 2E becomes 02
175 to 177	`mov al, cpu_ir` / `and al, 0Fh`	AND with 0F keeps only the low 4 bits, which is the **address**. Example: 2E becomes 0E
179 to 188	print the opcode and address	

On the 8086, a shift by more than 1 must use **CL** as the count. That's why we use `mov cl, 4`.

Lines 194 to 259 : EXECUTE

Lines 198 to 199 : `BX = op_addr`, the address that the instruction uses.

Lines 201 to 212 : Compare the opcode with 1 to 5 using `cmp` and `je` (jump if equal), and jump to the matching code. Anything else is treated as HALT.

Label	What it does	Key line
`do_load`	ACC ← M[address]	`mov cpu_acc, al`
`do_add`	ACC ← ACC + M[address]	`add cpu_acc, al`
`do_sub`	ACC ← ACC − M[address]	`sub cpu_acc, al`
`do_store`	M[address] ← ACC	`mov ram[bx], al`
`do_jmp`	PC ← address	`mov cpu_pc, bl`
`do_halt`	stop the CPU	`mov halted, al` (AL = 1)

The LOAD, ADD, SUB and STORE blocks also set **MAR = address** and **MDR = the data**, just like a real CPU.

Every block puts its message address in DX and jumps to **`exec_end`**, which prints the message and the registers (lines 256 to 259).

Lines 264 to 292 : `show_regs`

Prints `PC= MAR= MDR= IR= ACC=`. For each register it prints the label, then the value using `print_hex`.

Lines 297 to 316 : `show_memory`

A loop with **BX from 0 to 15**: print `ram[bx]` in hex, then a space. `inc bx` / `cmp bx, 16` / `jb mem_loop` repeats it 16 times.

Lines 323 to 361 : Helper procedures

Procedure	What it does
`print_str`	`mov ah, 9` / `int 21h`: prints the message whose address is in DX, up to the `$`
`wait_key`	`mov ah, 7` / `int 21h`: waits for a key press without showing the key
`print_hex`	Prints one byte as 2 hex digits: the high 4 bits first (`shr`), then the low 4 bits (`and 0Fh`)
`print_digit`	Turns 0 to 15 into the characters `'0'` to `'9'` or `'A'` to `'F'`, then prints it with `mov ah, 2` / `int 21h`

In `print_digit`, `'0'` is ASCII 48 and `'A'` is ASCII 65. For values 10 to 15, we add 7 extra to skip the 7 characters between `'9'` and `'A'`. `push` and `pop` save registers on the stack and restore them, so the procedure doesn't change values the caller still needs.

---

3. 8086 instructions used

Instruction	Use in this program
`MOV`	copy data between registers and variables
`LEA`	put the address of a message in DX
`ADD`, `SUB`	the ALU of the small CPU
`INC`	PC + 1, BX + 1
`SHR`	shift right 4 to get the opcode
`AND`	keep the low 4 bits to get the address
`CMP`, `JE`, `JNE`, `JB`, `JMP`	decisions and loops
`CALL`, `RET`	procedures
`PUSH`, `POP`	save and restore registers
`INT 21h`	DOS services: 09h prints a string, 02h prints a character, 07h reads a key, 4Ch exits

---

4. Questions sir may ask about the code

Why `org 100h`? It makes a .COM program, and DOS loads .COM programs at offset 100h.

Why `jmp start` at the top? The data comes first in the file. Without the jump, the CPU would try to run the data bytes as instructions.

Why `mov bh, 0` before `ram[bx]`? Only BX can be used as the index here, but our address is a byte in BL. Setting BH to 0 makes BX equal to BL.

Why `mov cl, 4` before `shr`? On the 8086, shifting by more than 1 needs the count in CL.

How do you get the opcode and the address from one byte?

- Opcode = `SHR` by 4, which keeps the high nibble.
- Address = `AND 0Fh`, which keeps the low nibble.
- Example: `3F` gives opcode 3 (SUB) and address F.

Where is the ALU? We use the 8086's own `ADD` and `SUB` on `cpu_acc`.

Why does every message end with `$`? INT 21h function 09h prints characters until it finds `$`.

What is `13, 10`? Carriage return and line feed, which start a new line.

What does `lea dx, msg` do? It loads the address of `msg` into DX. Function 09h prints the string at DS:DX.

How does the loop stop? `do_halt` sets `halted` to 1. The main loop checks it with `cmp al, 1` / `jne cpu_cycle`.

How does JMP change the program flow? `do_jmp` writes the address into `cpu_pc`. The next fetch reads from that address, so address 05 is skipped.

Why print in hex? Memory and addresses are normally shown in hex. Each byte is exactly 2 hex digits.

Can you change the program? Yes. Edit the bytes in the `ram` line. For example, changing the last three bytes changes the numbers being added and subtracted.